

ETRI UWB FirmWare User Guide

Version 1.0

Shmt Co., Ltd.

REVISION HISTORY PAGE

Issue	Rev No.	Originator	Details of Change	Date
1	0.1	DJKIM	Initial Version	2007-02-14
2	1.0	DJKIM	Modify HOWTO build execute file	2007-03-26
3				
4				
5				

Confidential

Technical documents for Firmware

Copyright © 2007 Shmt Limited. All rights reserved.

Confidentiality Status

This document is **NOT OPEN ACCESS**. The distribution is restricted to certified company.
This document is **CONFIDENTIAL**.

Feedback on this document

If you have any comments on this document, please contact your channel to our company.

Shmt Web address

<http://www.shmt.com>

Confidential

Contents

1. INTRODUCTION	3
2. OverView	3
3. SW Hierarchy	3
4. Directory Analysis	3
4.1. main	3
4.1.1. app	3
4.1.2. app/nand_boot	3
4.1.3. startup	3
4.2. sys	3
4.3. peri	3
4.4. inc	3
5. HOWTO build execute file	3
5.1. ADS v1.2 Using	3
5.1.1. FirmWare Project Structure	3
6. HOWTO Porting	3
6.1. Memory map	3
6.2. System setting	3
6.3. Source modification	3

Figures

Confidential

Tables

Confidential

1. INTRODUCTION

본 문서는 가라오케 시스템을 위한 low-cost solution SOC로 개발된 ETRI UWB의 firmware에 대한 내용을 다룬다. H/W 설계 과정 및 FPGA 검증 과정에서 사용되는 firmware의 내용과 디렉토리 및 S/W hierarchy가 설명되었다.

Confidential

2. OverView

ETRI UWB은 ARM926EJ-S을 내장하고 있으며 다양한 어플리케이션에 효율적으로 대응할 수 있는 SOC 칩으로 다음과 같은 features를 갖는다.

- ARM926EJ core

Normal operation mode : 100MHz

2x operation mode : 200MHz

- Video Encoder

NTSC/PAL display

One DAC for CVBS, two times over-sampled with 10 bit resolution.

Internal Color Bar Generator.

Cross color reduction filter.

- Internal PSRAM

4 MBits

Optional configuration

- UART 4 Channel

UART 32 depth FIFO 내장한 버전 2 ch

UART 8 depth FIFO 내장한 버전 2 ch

- Graphic Engine

NTSC/PAL output mode : 704 x 480i / 704 x 576i (max)

VGA output mode : 640 x 480p(VGA), 320x240p(QVGA)

4pages

- Wide(1440x576) x 2page : no Palette

- VGA (640x576) x 2page : Palette 256 color/page

Overlay & α-blending : 3pages(Wide 1 page + VGA 2pages).

Support chroma keying : Color selectable.

Line Up-scaling : 480 -> 576 (NTSC -> PAL).

Up-scaling support function

For helping NTSC->PAL conversion.

Insert 1 horizontal line, copied from the 5th line, each 5 lines.

No line filter required.

Configurable Outputs : RGB(565) & Sync/ITU-R BT.601(16bit)/ITU-R BT.656(8bit).

Internal Color Bar Generator.

Color Space Conversion.

■ DDR Controller

128/256/512 Mbit

■ I2C

Variable clock rate : 400kHz max.

■ I2S

Configurable Master/Slave.

ADC(receive) & DAC(transmit) IF for audio Codec.

IF mode : Standard I₂S, MSB(=Left) justified mode(configurable).

8/16/24/32bit Word lengths.

16k/32k/44.1k/48kHz Sampling Frequency.

Each 16 word FIFO entries for Transmit & Receive.

■ Timer

■ Enhanced-DMA

■ Graphic DMA

■ SPI

SPI clock speed up to 12MHz.

Ch-1 : For transferring MP3 bit-stream.

160Byte/10mS @128k Sampling.

Support DMA.

Ch-2 : For transferring commands.

Data Length(max) : 64bit(8byte).

No needs DMA.

■ UART

Special UART (2ch)

Baudrate : 115200 max

Tx 32bytes & Rx 32bytes FIFOs.

31.25k baudrate support for MIDI.

Standard UART (2ch)

Baudrate : 115200 max

Tx 8bytes & Rx 8bytes FIFOs.

- Multi-boot

For support wide range application

- Sound Engine

Mask rom for Sound Engine : 32 / 64 / 128 MBits

Internal SRAM for Sound Engine : 1 MBits + 24K bits

Operation Clock : 100MHz.

(128K + 3072)Byte SRAM

Sound effect & MIC echo memory.

512Byte ROM.

Audio CODEC : Excluded.

Wave Playback & Recorder Interface.

1024 point FFT with I2S receiver.

4ch I2S Master interface : 2ch ADC in & 2ch DAC out .

PCM Rx/Tx interface for Record & Play.

- NAND Flash memory controller

32M ~ 2G Byte.

512/256 byte, 1-bit ECC.

Command FIFO/Data FIFO.

Support small/large block.

Support SLC/MLC.

- USB 2.0

USB 1.1/2.0 compliant.

PHY included.

- External Video Input

Digital Video Decoder Interface : ITU-R BT.656(8bit).

Supports Interlace / Non-interlace Mode.

Color Space Conversion.

Memory Buffering.

Support Frame Capture.

- Extra Asynchronous Chip select signal for reservation : 4 (8/16bit).

Async IF – ex) ROM/Nor-flash/SRAM IF.

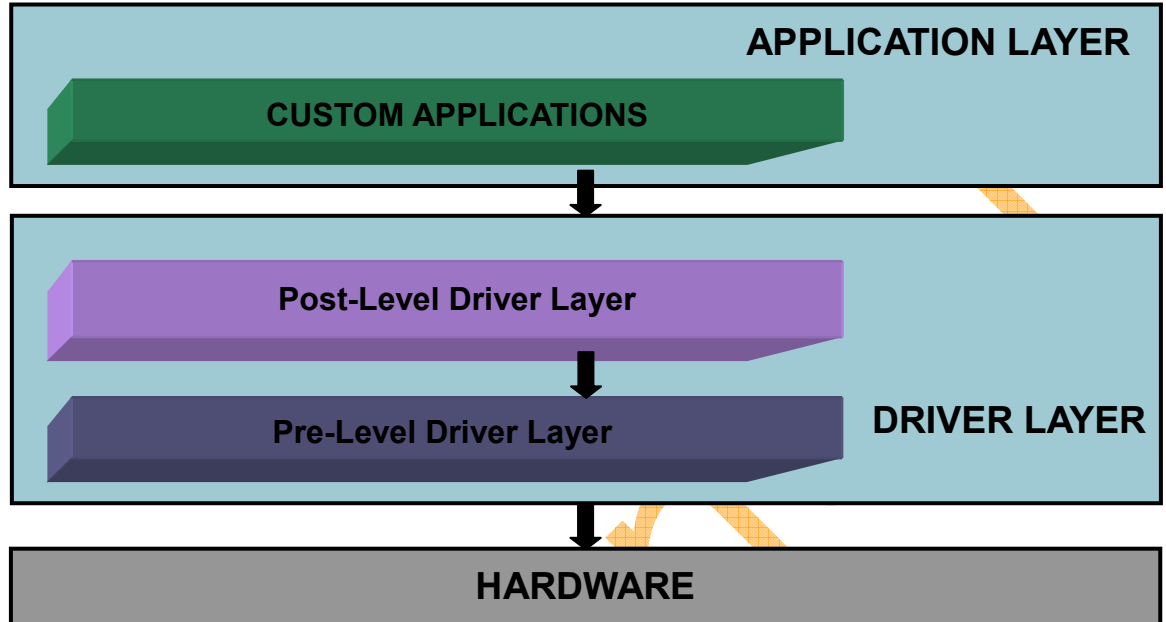
Wait configurable.

Upto 128M Byte.

Confidential

3. SW Hierarchy

다음 그림은 ETRI UWB의 firmware S/W hierarchy를 나타낸다.



[Fig - 1] FirmWare S/W Hierarchy

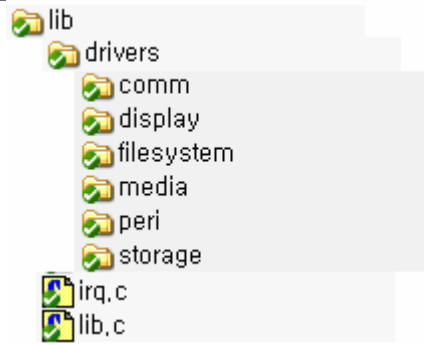
ETRI UWB의 H/W를 firmware로 구성함에 있어 S/W firmware layer는 세가지로 구분하였다.

1) Pre-level driver layer

기본 peri(Memory, Timer/PWM, WDT, GPIO, VIC, ADC, Power management etc.)와 통신(UART, I2C, I2S, SPI etc), 디스플레이(LCD, Video Encoder), 미디어(AUDIO), 스토리지(NAND)의 register level 동작을 조작한다.

기본 peri에 대한 pre-level driver 함수는 lib/, lib/drivers/peri/ 디렉토리에 peri 모듈 별 파일로 구성이 되어 있으며, 함수명은 `smtfuncName()`로 지정된다.

통신, 디스플레이, 미디어, 스토리지의 모듈 별 파일 구성도 이와 비슷하다. Pre-level driver 함수들은 `module_pre_drv.c` 파일로 존재하며 다음은 pre-level driver 함수의 디렉토리 구조를 나타낸다.



[Fig - 2] Pre-level driver directory tree

2) Post-level drivers layer

Pre-level driver layer보다 상위 driver layer로 pre-driver function을 호출하여 scenario적인 관점의 동작을 조작한다. Pre-driver 함수의 경우 register의 셋팅과 상태 체크 등 레지스터 레벨의 함수인 반면 post-driver 함수는 pre-driver 함수를 사용하여 read/write 등과 같은 scenario 동작과 관련된다.

Post-level driver 함수의 경우 pre-level driver 함수와 달리 함수명이 smt2 로 시작한다. Post-level driver 함수들은 pre-level driver 함수들과 같은 디렉토리에 module_post_drv.c 파일로 존재한다.

4. Directory Analysis

상화에서 제공하는 F/W 프로그램의 디렉토리는 다음과 같다.

1) inc/ directory

Driver 함수들의 선언과 사용되는 매크로, 전역 변수들을 담은 헤더 파일들이 있다.

drivers/

comm./

통신과 관련된 디바이스들(I2C, I2S, SPI, UART, USB etc)의 pre/post driver 헤더 파일들이 있다.

display/

이미지 디스플레이 디바이스(LCD, Video Encoder)의 pre/post driver 헤더 파일들이 있다.

filesystem/

파일 시스템을 위한 FAT32, partition 관련 헤더 파일이 있다.

media/

미디어(AUDIO) 디바이스의 H/W, S/W pre/post driver 헤더 파일이 있다.

peri/

storage/

Storage (NAND, SD/MMC) 디바이스의 pre/post driver 헤더파일이 있다.

fw_drv_err/

Post-level driver 함수의 에러 리턴을 위한 enum 정의 헤더파일이 있다.

commonmacro.h

- ETRI UWB F/W에서 사용할 수 있는 일반적인 매크로를 정의

global.h

- ETRI UWB F/W에서 사용할 수 있는 전역 변수의 정의

irq.h

- VIC를 위한 매크로와 함수 선언을 포함

lib.h

- Basic peripheral의 매크로 정의와 함수 선언을 포함

structdef.h

- ETRI UWB F/W에서 사용되는 데이터 타입을 정의

2) lib/ directory

Peripheral의 driver 함수들이 정의 되어 있다.

drivers/

ETRI UWB에서 사용한 pre/post-level driver 함수들이 정의 되어 있다.

comm./

I2C, I2S, UART, USB의 pre/post-level driver 함수 정의

display/

LCD, Video Encoder의 pre/post-level driver 함수 정의

filesystem/

FAT32 파일 시스템 post-level driver 함수 정의

media/

AUDIO의 post-level driver 및 mp3/ogg post-level driver 함수 정의

peri/

storage/

NAND, SD/MMC의 pre/post-level driver 함수 정의

irq.c

VIC 관련 pre/post-level driver 함수 정의

lib.c

Basic peripheral의 pre/post-level driver 함수 정의

3) main/ directory

- app/

User의 F/W 소스 및 application 소스가 위치하는 디렉토리

- bootloader/

NAND/NOR booting을 지원하기 위한 boot-loader 소스가 존재 (구축 준비 중)

- startup/

FirmWare main 함수로 진입하기 위해 필요한 ARM926 의 startup code와 memory configuration을 포함한다. (ADSV1.2 사용 할 경우 - [5.2 ADS v1.2 Using](#) 참조)

- startup-net/

- main.c

F/W 코드의 메인 함수

- mmu.c

ARM926EJ-S의 mmu setting을 위한 함수

- regview_v2.c

디버거 장비를 통한 디버깅 시 ETRI UWB의 레지스터 맵을 watch 할 수 있도록 레지스터 구조체 정의

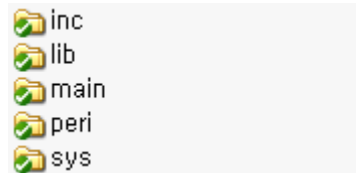
4) peri/ directory

각 peri 별 FPGA 테스트 파일과 헤더 파일이 존재한다.

5) sys/ directory

ETRI UWB의 system 레벨에서의 설정이 이루어진다. Memory 사이즈, clock 설정, boot mode 설정, memory bus width 등을 설정한다.

[Fig- 4]는 디렉토리 구조를 나타낸다.



[Fig - 3] F/W directory tree

Driver 함수에 대한 자세한 사항은 programming guide를 참조하기 바라며, 여기서는 스타트업 코드와 상화마이크로텍에서 제공하는 F/W 코드에서 공통적으로 사용하는 system configuration 관련 매크로 및 데이터 타입 정의에 대해서 설명하기로 한다.

4.1. main

4.1.1. app

ETRI UWB의 F/W 어플리케이션 소스가 있다.

4.1.2. app/nand_boot

NAND booting에 사용되는 부트 로더가 있다.

4.1.3. startup

Startup 디렉토리의 파일들은 처음 reset 후 ETRI UWB의 동작 시 main() 함수로 진입하기 전 이루어지는 초기화 함수들이 포함된다.

다음은 startup directory에 있는 파일들에 대한 설명이다.

1) Startup.s

ARM926EJ core startup source file

2) smt926addr.inc

Startup.s에서 사용되는 memory bank configuration variable의 값을 각 bank별로

설정한다. 또한 bus, pll, clock configuration variable의 값을 설정한다.

3) asmlib.s

ARM926EJ-S core의 irq mode en/disable 어셈블리 함수가 있다.

4) mmufunc.s

ARM926EJ-S core의 MMU I/D-TLB의 clear assembly 함수가 있다.

4.2. sys

sys directory의 파일들은 ETRI UWB의 firmware 동작을 위한 system 관련 설정 헤더 파일들이다.

1) sysinc.h

Sys directory의 모든 헤더 파일을 포함한다.

2) sysdatadef.h

ETRI UWB의 firmware에서 사용되는 data type을 정의한다.

3) sysreg.h

ETRI UWB의 register map, 각 register 별로 할당된 비트 마스크를 정의한다.

4) syscfg.h

ETRI UWB의 boot mode, flash memory size 등 system에 관련 설정을 정의한다.

4.3. peri

peri 디렉토리의 파일들은 각 H/W 모듈 테스트 함수가 있는 파일들이다. 다음에서 설명하는 각 H/W 모듈들의 설명은 알파벳 순서에 따르도록 한다.

1) DMA

E-DMA(Enhanced)와 G-DMA(Graphic-DMA)로 구성되어 있으며, E-DMA는 memory to I/O, I/O to memory, memory to memory의 데이터 복사를 가능하게 한다.

G-DMA는 메모리에서 일정한 2 차원 영역을 다른 메모리 어드레스의 2 차원 영역으로 복사하는 일을 수행한다.

2) GPIO

3) TIMER

Timer는 interval, match & overflow 모드를 지원한다.

4) WDT

5) VIC

6) Communication Modules

가) I2C

나) I2S

다) SPI

마) UART

7) Storage

가) Nand Flash

나) SRAM

다) DDR-SDRAM

4.4. inc

ETRI UWB의 pre/post-level driver에서 함수 구현을 위해 사용하는 헤더파일이 있다. Driver 함수들을 위해서 서브 디렉토리 별로 pre/post-level driver 헤더파일들이 있다. 또한 basic peripheral들을 위한 헤더파일(lib.h, irq.h)과 common 매크로 및 basic peripheral의 프로그램 구조체가 정의된 헤더파일(commonmacro.h, structdef.h)가 있다.

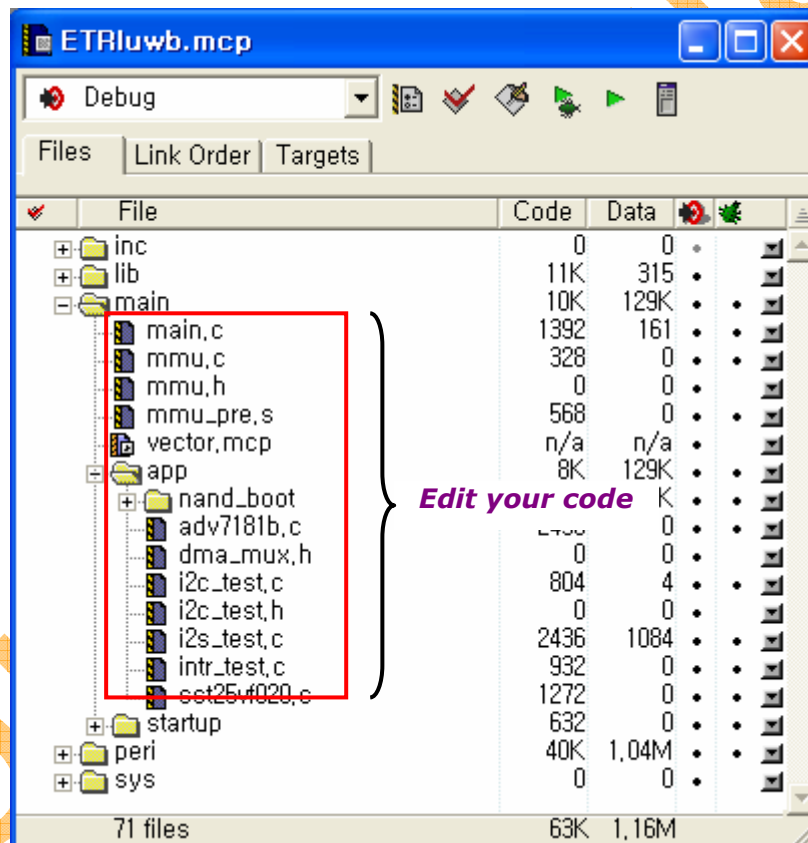
5. HOWTO build execute file

F/W의 실행 파일 생성은 windows 상에서 ADSv1.2 IDE를 사용하여 생성할 수 있도록 구성되어 있다.

5.1. ADS v1.2 Using

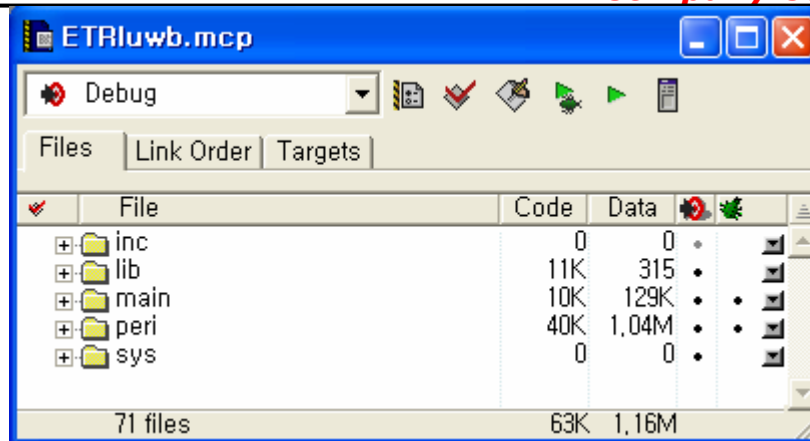
5.1.1. FirmWare Project Structure

다음은 ADSv1.2 를 사용하여 구성한 ETRI UWB의 firmware project structure이다.

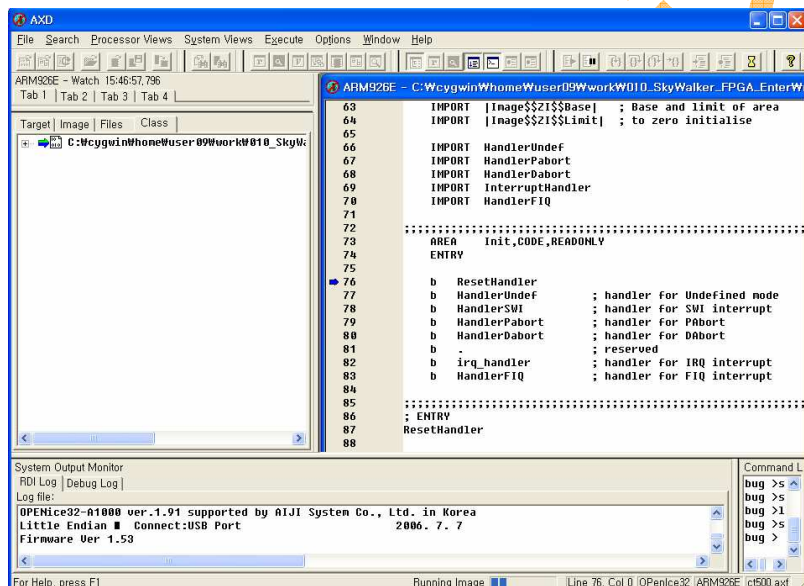


[Fig - 4] ADSv1.2 Project Structure

응용프로그램 개발 시 a1000 과 같은 디버거 장비를 사용할 경우는 ETRI UWB 프로젝트 (ETRI UWB.mcp)에서 debug mode상에서 실행 파일을 생성하고 디버거 스크립트를 활용하여 axd 또는 aiji의 디버거 프로그램 spider를 사용하면 된다. [Fig - 5]과 [Fig - 6]은 실행 파일 생성과 디버거 사용 시 axd 상에서의 디버깅을 보여준다.

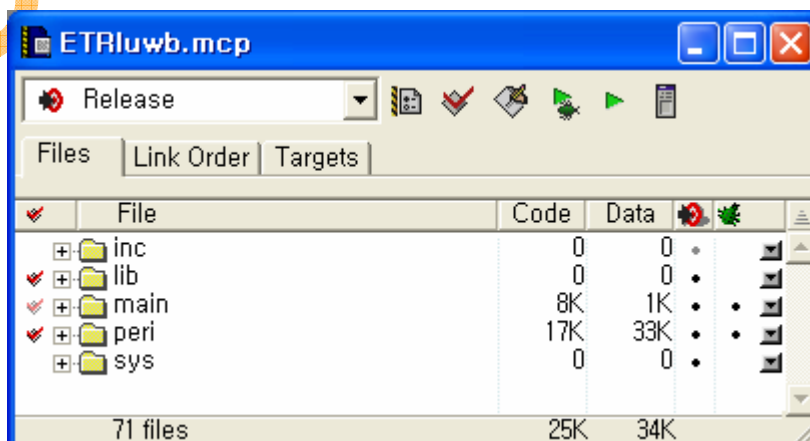


[Fig - 5] Debug 모드에서 실행 파일 생성 시

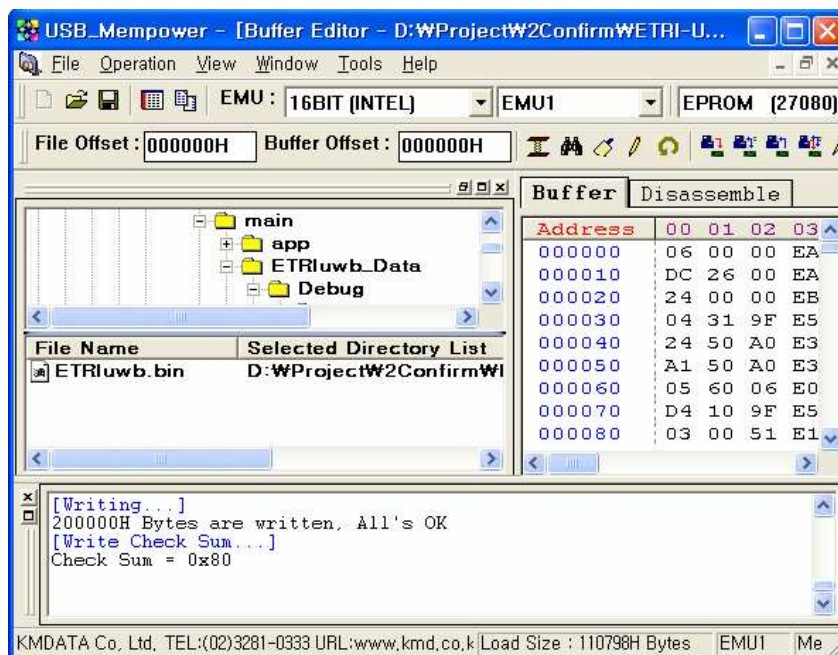


[Fig - 6] axd 를 사용한 디버깅 시

롬 에뮬레이터(External memory Select0)를 사용하는 경우는 Release 모드에서 binary 파일을 생성하고 binary를 다운로드하여 실행하면 된다.



[Fig - 7] Release 모드에서 실행 파일 생성 시



[Fig - 8] 롬에물레이터 사용 시

6. HOWTO Porting

상화에서 제공되는 ETRI UWB용 F/W 소스를 해당 어플리케이션에 맞게 포팅하기 위해서는 다음과 같이 하면 된다.

- 1) 메모리 맵과 스택 영역에 맞추어 **startup** 코드를 수정한다.

개발자의 특별한 요구 사항과 메모리 맵의 수정이 필요한 경우를 제외하고는 대개의 경우 상화 제공 **startup** 코드를 그대로 사용하면 된다.

main/startup/startup.s

- 2) 개발 어플리케이션에 맞게 소스를 작성한다.

main/mmu.c

main/app

응용프로그램을 개발 시 [Fig - 4]에서와 같이 **main/** 디렉토리 부분에서만 수정이 이루어지면 된다. 응용프로그램을 호출하는 **main()** 함수와 해당 응용프로그램 함수의 구현이 이루어지는 **app** 디렉토리가 그 부분이다.

6.1. Memory map

상화에서 제공되는 F/W 소스의 메모리 맵은 **binary** 실행 파일을 사용하는 **ROM base**일 때와 디버거를 사용하는 **DDR base**로 각각의 메모리 맵은 다음과 같다.

- 1) ROM emulator Base (ROM / NOR booting 모드)

: ROM emulator가 external memory select0 영역에, NOR flash가 external memory select1 영역에 있을 경우

- ROM 영역

- Code 영역 : 0x00000000 ~ 0x00200000

F/W 코드가 위치하는 영역

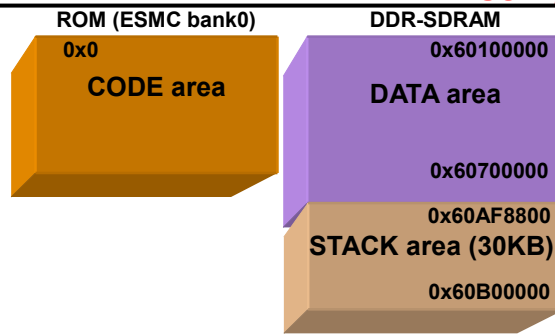
- DDR-SDRAM 영역

- Data 영역 : 0x60100000 ~ 0x60600000

전역변수 및 **static** 변수가 위치하는 영역

- Stack 영역 : 0x60AF8800 ~ 0x60B00000

Exception 모드(User/SVC/IRQ/FIQ/ABT/UND) 별 스택 영역

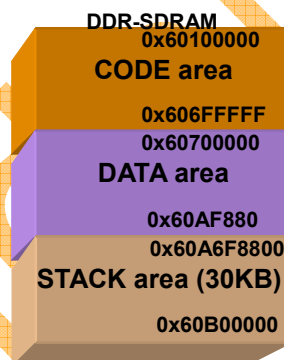


[Fig - 9] ROM base memory map

2) DDR-SDRAM Base (Debugger / NAND booting 모드)

: A1000 과 같은 디버거로 DDR-SDRAM 영역을 사용, NOR flash가 external memory select1 영역에 있을 경우

: NAND booting을 할 경우 NAND의 bootloader에 의해 F/W code는 nand 영역에서 DDR의 CODE area 영역으로 복사되어 DDR-SDRAM base의 메모리 맵과 동일하게 구성되어 동작한다.



[Fig - 10] DDR-SDRAM base memory map

6.2. System setting

개발하고자 하는 제품에 맞도록 syscfg.h의 시스템 설정을 수정하면 된다. 그러나 특별한 경우를 제외하고 대개의 경우 상화에서 제공하는 플랫폼의 시스템 설정을 그대로 사용하면 된다.

6.3. Source modification

1) 메모리 맵과 스택 영역에 맞추어 startup 코드 수정

개발하고자 하는 응용프로그램의 메모리 맵을 위한 수정이 필요한 경우를 제외하고는 startup과 메모리 맵은 수정하지 않아도 된다.

2) ROM base와 DDR-SDRAM base에 따라서 mmu.c 수정

- MMU_Init() 함수에서 다음 부분을 수정한다.

#if 1 // DDR-SDRAM base 일 경우 활성화

MMU_SetMTT(0x62000000,0x63F00000,0x62000000,RW_CNB); // DDR 15MByte

MMU_SetMTT(0x60100000,0x61F00000,0x60100000,RW_CNB); // DDR 48MByte

MMU_SetMTT(0x00000000,0x00100000,0x60000000,RW_CNB); // DDR 1MByte

#else // ROM base 일 경우 활성화

MMU_SetMTT(0x62000000,0x63F00000,0x62000000,RW_NCNB); // DDR 32MByte

MMU_SetMTT(0x60000000,0x61F00000,0x60000000,RW_CNB); // DDR 32MByte

MMU_SetMTT(0x00000000,0x00200000,0x00000000,RW_CNB); // DDR 2MByte

#endif

3) 응용 프로그램에 맞게 main/app 디렉토리에 소스를 작성